

VER-2026-001

Cryptographic Compliance Verification

SendCUIEmail v0.17.3

Verification Document

January 22, 2026

Document Information

Document ID:	VER-2026-001
Subject:	Cryptographic Implementation Compliance Verification
Requirements:	REQ-2026-001 v1.1
Software Version:	v0.17.3
Git Commit:	2048d2e4508e83ca77b264949775835a3d3aba96
Commit Date:	2026-01-22 10:12:44 -0600
Source File:	Encrypt.ps1

Executive Summary

This document verifies SendCUIEmail’s compliance with REQ-2026-001 Cryptographic Compliance Requirements. Each requirement is traced by its REQ ID, with code evidence demonstrating compliance.

Contents

1	Encryption Algorithm Requirements	2
1.1	REQ-1.1: AES Algorithm	2
1.2	REQ-1.2: 256-bit Key Length	2
1.3	REQ-1.3: CBC Mode	2
1.4	REQ-1.4: Unique Random IV	2
1.5	REQ-1.5: IV Size	3
1.6	REQ-1.6: IV Storage	3
2	Key Derivation Requirements	4
2.1	REQ-2.1: PBKDF2	4
2.2	REQ-2.2: HMAC-SHA256 PRF	4
2.3	REQ-2.3: Iteration Count	4
2.4	REQ-2.4: Unique Random Salt	4
2.5	REQ-2.5: Salt Size	5
2.6	REQ-2.6: Salt Storage	5
2.7	REQ-2.7: Iteration Count Review (Recommended)	5

3	Random Number Generation Requirements	6
3.1	REQ-3.1: CSPRNG	6
3.2	REQ-3.2: Windows System RNG	6
3.3	REQ-3.3: NIST SP 800-90A Compliance	6
4	Password Requirements	7
4.1	REQ-4.1: Minimum Password Length	7
4.2	REQ-4.2: Password Confirmation	7
4.3	REQ-4.3: Auto Passphrase Generation (Recommended)	7
4.4	REQ-4.4: No Password Storage	8
4.5	REQ-4.5: Masked Password Input	8
5	File Format Requirements	9
5.1	REQ-5.1: File Contents	9
5.2	REQ-5.2: File Format	9
5.3	REQ-5.3: File Extension	9
5.4	REQ-5.4: Original Extension Preserved	9
6	Platform Requirements	10
6.1	REQ-6.1: Platform Crypto Libraries	10
6.2	REQ-6.2: .NET Cryptographic Classes	10
6.3	REQ-6.3: FIPS Mode Warning (Recommended)	10
6.4	REQ-6.4: No Third-Party Software	10
7	Verification Summary	12
8	Conclusion	12

1 Encryption Algorithm Requirements

1.1 REQ-1.1: AES Algorithm

Requirement: The system SHALL use AES (Advanced Encryption Standard) as defined in FIPS 197 for symmetric encryption.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 347):

```
1 $aes = [System.Security.Cryptography.Aes]::Create()
```

Result: PASS ✓ — Uses .NET Aes class which implements FIPS 197.

1.2 REQ-1.2: 256-bit Key Length

Requirement: The system SHALL use a key length of 256 bits (AES-256) for all encryption operations.

Verification Method: Inspection

Evidence (Encrypt.ps1, lines 61, 344):

```
1 $KEY_SIZE = 32          # bytes (256 bits for AES-256)
2 ...
3 $key = $keyDeriver.GetBytes($KEY_SIZE)
```

Result: PASS ✓ — Key is 256 bits (32 bytes).

1.3 REQ-1.3: CBC Mode

Requirement: The system SHALL use Cipher Block Chaining (CBC) mode or an authenticated encryption mode (GCM, CCM).

Verification Method: Inspection

Evidence (Encrypt.ps1, line 348):

```
1 $aes.Mode = [System.Security.Cryptography.CipherMode]::CBC
```

Result: PASS ✓ — Uses CBC mode.

1.4 REQ-1.4: Unique Random IV

Requirement: The system SHALL generate a unique, random Initialization Vector (IV) for each encryption operation.

Verification Method: Test

Evidence (Encrypt.ps1, lines 331-335):

```
1 $rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
2 $salt = New-Object byte[] $SALT_SIZE
3 $iv = New-Object byte[] $IV_SIZE
```

```
4 $rng.GetBytes($salt)
5 $rng.GetBytes($iv)
```

Test: Encrypted same file twice, verified IVs differ in output.

Result: PASS ✓ — New random IV generated for each encryption.

1.5 REQ-1.5: IV Size

Requirement: The IV SHALL be at least 128 bits (16 bytes) for AES-CBC mode.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 62):

```
1 $IV_SIZE = 16           # bytes (128 bits for AES block size)
```

Result: PASS ✓ — IV is exactly 128 bits.

1.6 REQ-1.6: IV Storage

Requirement: The IV SHALL be stored with or transmitted alongside the ciphertext.

Verification Method: Inspection

Evidence (Encrypt.ps1, lines 358-361):

```
1 # Combine: Salt + IV + Ciphertext
2 $outputBytes = New-Object byte[] ($SALT_SIZE + $IV_SIZE + $cipherBytes.
   Length)
3 [Array]::Copy($salt, 0, $outputBytes, 0, $SALT_SIZE)
4 [Array]::Copy($iv, 0, $outputBytes, $SALT_SIZE, $IV_SIZE)
```

Result: PASS ✓ — IV stored at bytes 16-31 of output file.

2 Key Derivation Requirements

2.1 REQ-2.1: PBKDF2

Requirement: The system SHALL use PBKDF2 (Password-Based Key Derivation Function 2) as defined in NIST SP 800-132 for deriving encryption keys from passwords.

Verification Method: Inspection

Evidence (Encrypt.ps1, lines 338-343):

```
1 $keyDeriver = New-Object System.Security.Cryptography.  
    Rfc2898DeriveBytes(  
2     $Password,  
3     $salt,  
4     $ITERATIONS,  
5     [System.Security.Cryptography.HashAlgorithmName]::SHA256  
6 )
```

Result: PASS ✓ — Uses Rfc2898DeriveBytes (PBKDF2 implementation).

2.2 REQ-2.2: HMAC-SHA256 PRF

Requirement: The system SHALL use HMAC-SHA256 as the pseudorandom function (PRF) for PBKDF2.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 342):

```
1 [System.Security.Cryptography.HashAlgorithmName]::SHA256
```

Result: PASS ✓ — Explicitly specifies SHA256 for HMAC.

2.3 REQ-2.3: Iteration Count

Requirement: The system SHALL use a minimum of 100,000 iterations for PBKDF2 per NIST SP 800-132 recommendations.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 59):

```
1 $ITERATIONS = 100000 # PBKDF2 iterations
```

Result: PASS ✓ — Uses exactly 100,000 iterations.

2.4 REQ-2.4: Unique Random Salt

Requirement: The system SHALL generate a unique, random salt for each encryption operation.

Verification Method: Test

Evidence (Encrypt.ps1, lines 331-334):

```
1 $rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
2 $salt = New-Object byte[] $SALT_SIZE
3 ...
4 $rng.GetBytes($salt)
```

Test: Encrypted same file twice with same password, verified salts differ.

Result: PASS ✓ — New random salt generated for each encryption.

2.5 REQ-2.5: Salt Size

Requirement: The salt SHALL be at least 128 bits (16 bytes) as recommended by NIST SP 800-132.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 60):

```
1 $SALT_SIZE = 16          # bytes (128 bits)
```

Result: PASS ✓ — Salt is exactly 128 bits.

2.6 REQ-2.6: Salt Storage

Requirement: The salt SHALL be stored with or transmitted alongside the ciphertext.

Verification Method: Inspection

Evidence (Encrypt.ps1, lines 358-359):

```
1 $outputBytes = New-Object byte[] ($SALT_SIZE + $IV_SIZE + $cipherBytes.
    Length)
2 [Array]::Copy($salt, 0, $outputBytes, 0, $SALT_SIZE)
```

Result: PASS ✓ — Salt stored at bytes 0-15 of output file.

2.7 REQ-2.7: Iteration Count Review (Recommended)

Requirement: The iteration count SHOULD be increased over time as computing power increases.

Verification Method: Analysis

Analysis: Current iteration count (100,000) was set in 2026. NIST recommends reviewing iteration counts periodically. The constant \$ITERATIONS is centrally defined for easy updates.

Result: PASS ✓ — Iteration count is configurable and documented.

3 Random Number Generation Requirements

3.1 REQ-3.1: CSPRNG

Requirement: The system SHALL use a cryptographically secure random number generator (CSPRNG) for generating salts, IVs, and other cryptographic material.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 331):

```
1 $rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
```

Result: PASS ✓ — Uses .NET RandomNumberGenerator (CSPRNG).

3.2 REQ-3.2: Windows System RNG

Requirement: On Windows, the system SHALL use the system RNG provided by .NET (RNGCryptoServiceProvider or RandomNumberGenerator).

Verification Method: Inspection

Evidence: Same as REQ-3.1. RandomNumberGenerator.Create() returns the platform-appropriate CSPRNG (Windows CNG on Windows).

Result: PASS ✓

3.3 REQ-3.3: NIST SP 800-90A Compliance

Requirement: The CSPRNG SHALL meet the requirements of NIST SP 800-90A or equivalent.

Verification Method: Analysis

Analysis: Windows CNG (bcryptprimitives.dll) is FIPS 140-2 validated (CMVP #4515) and implements NIST SP 800-90A compliant random number generation.

Result: PASS ✓

4 Password Requirements

4.1 REQ-4.1: Minimum Password Length

Requirement: The system SHALL enforce a minimum password length of 8 characters.

Verification Method: Test

Evidence (Encrypt.ps1, lines 50, 269-272):

```
1 $MIN_PASSWORD_LENGTH = 8
2 ...
3 if ($plain.Length -lt $MIN_PASSWORD_LENGTH) {
4     Write-Host "Password must be at least $MIN_PASSWORD_LENGTH
5     characters"
6     ...
```

Test: Attempted encryption with 7-character password; rejected.

Result: PASS ✓

4.2 REQ-4.2: Password Confirmation

Requirement: The system SHALL require password confirmation during encryption to prevent typos.

Verification Method: Test

Evidence (Encrypt.ps1, lines 280-293):

```
1 $confirm = Read-Host "Confirm password" -AsSecureString
2 ...
3 if ($plain -ne $confirmPlain) {
4     Write-Host "Passwords do not match!" -ForegroundColor Red
```

Test: Entered mismatched passwords; rejected with error message.

Result: PASS ✓

4.3 REQ-4.3: Auto Passphrase Generation (Recommended)

Requirement: The system SHOULD offer automatic passphrase generation using cryptographic RNG.

Verification Method: Test

Evidence (Encrypt.ps1, lines 230-244, 258-262):

```
1 function New-Passphrase {
2     param([int]$WordCount = 4)
3     $rng = [System.Security.Cryptography.RandomNumberGenerator]::Create
4     ...
```

Test: Pressed Enter at password prompt; received random passphrase.

Result: PASS ✓

4.4 REQ-4.4: No Password Storage

Requirement: Passwords SHALL NOT be stored or logged by the system.

Verification Method: Inspection

Evidence: Code review confirms password is only held in memory during encryption, then discarded. No file writes or logging of password values.

Result: PASS ✓

4.5 REQ-4.5: Masked Password Input

Requirement: Password input SHALL be masked (not displayed on screen) during entry.

Verification Method: Test

Evidence (Encrypt.ps1, line 267):

```
1 $password = Read-Host "Enter encryption password" -AsSecureString
```

Test: Typed password; characters not displayed (asterisks not shown either per SecureString behavior).

Result: PASS ✓

5 File Format Requirements

5.1 REQ-5.1: File Contents

Requirement: Encrypted files SHALL contain: salt, IV, and ciphertext in a defined format.

Verification Method: Inspection

Evidence: See REQ-5.2 for format specification.

Result: PASS ✓

5.2 REQ-5.2: File Format

Requirement: The file format SHALL be: [16-byte salt][16-byte IV][ciphertext].

Verification Method: Test

Evidence (Encrypt.ps1, lines 358-361):

```
1 $outputBytes = New-Object byte[] ($SALT_SIZE + $IV_SIZE + $cipherBytes.  
    Length)  
2 [Array]::Copy($salt, 0, $outputBytes, 0, $SALT_SIZE)  
3 [Array]::Copy($iv, 0, $outputBytes, $SALT_SIZE, $IV_SIZE)  
4 [Array]::Copy($cipherBytes, 0, $outputBytes, $SALT_SIZE + $IV_SIZE,  
    ...)
```

Test: Hex dump of .Locked file shows 16-byte salt, 16-byte IV, then ciphertext.

Result: PASS ✓

5.3 REQ-5.3: File Extension

Requirement: Encrypted files SHALL use the .Locked extension to indicate encrypted status.

Verification Method: Inspection

Evidence (Encrypt.ps1, line 364):

```
1 $outputPath = "$InputPath.Locked"
```

Result: PASS ✓

5.4 REQ-5.4: Original Extension Preserved

Requirement: The original file extension SHALL be preserved (e.g., document.pdf becomes document.pdf.Locked).

Verification Method: Test

Test: Encrypted test.pdf; output was test.pdf.Locked.

Result: PASS ✓

6 Platform Requirements

6.1 REQ-6.1: Platform Crypto Libraries

Requirement: The system SHALL use platform-provided cryptographic libraries (no custom implementations).

Verification Method: Inspection

Evidence: All cryptographic operations use `System.Security.Cryptography` namespace classes. No custom algorithm implementations.

Result: PASS ✓

6.2 REQ-6.2: .NET Cryptographic Classes

Requirement: On Windows, the system SHALL use .NET Framework cryptographic classes (`System.Security.Cryptography`).

Verification Method: Inspection

Evidence: Uses:

- `System.Security.Cryptography.Aes`
- `System.Security.Cryptography.Rfc2898DeriveBytes`
- `System.Security.Cryptography.RandomNumberGenerator`

Result: PASS ✓

6.3 REQ-6.3: FIPS Mode Warning (Recommended)

Requirement: The system SHOULD detect and warn if FIPS mode is not enabled on Windows.

Verification Method: Test

Evidence (Encrypt.ps1, lines 76-108):

```
1 function Test-FIPSMode {  
2     ...  
3     $fipsKey = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\  
4         FIPSAAlgorithmPolicy"  
5     ...  
6 }
```

Test: Ran on system with FIPS mode disabled; warning displayed.

Result: PASS ✓

6.4 REQ-6.4: No Third-Party Software

Requirement: The system SHALL NOT require installation of third-party software on recipient systems.

Verification Method: Analysis

Analysis: Recipients only need Windows PowerShell (built-in) to decrypt files. The `Decrypt_Instructions.html` contains a one-liner that works on any Windows system without installation.

Result: PASS ✓

7 Verification Summary

REQ ID	Description	Method	Status
REQ-1.1	AES Algorithm	Inspection	✓
REQ-1.2	256-bit Key	Inspection	✓
REQ-1.3	CBC Mode	Inspection	✓
REQ-1.4	Unique Random IV	Test	✓
REQ-1.5	IV Size (128-bit)	Inspection	✓
REQ-1.6	IV Storage	Inspection	✓
REQ-2.1	PBKDF2	Inspection	✓
REQ-2.2	HMAC-SHA256	Inspection	✓
REQ-2.3	100k Iterations	Inspection	✓
REQ-2.4	Unique Random Salt	Test	✓
REQ-2.5	Salt Size (128-bit)	Inspection	✓
REQ-2.6	Salt Storage	Inspection	✓
REQ-2.7	Iteration Review	Analysis	✓
REQ-3.1	CSPRNG	Inspection	✓
REQ-3.2	Windows RNG	Inspection	✓
REQ-3.3	NIST SP 800-90A	Analysis	✓
REQ-4.1	Min Password Length	Test	✓
REQ-4.2	Password Confirmation	Test	✓
REQ-4.3	Auto Passphrase	Test	✓
REQ-4.4	No Password Storage	Inspection	✓
REQ-4.5	Masked Input	Test	✓
REQ-5.1	File Contents	Inspection	✓
REQ-5.2	File Format	Test	✓
REQ-5.3	.Locked Extension	Inspection	✓
REQ-5.4	Extension Preserved	Test	✓
REQ-6.1	Platform Libraries	Inspection	✓
REQ-6.2	.NET Crypto Classes	Inspection	✓
REQ-6.3	FIPS Mode Warning	Test	✓
REQ-6.4	No Third-Party SW	Analysis	✓

Summary: All 28 requirements from REQ-2026-001 v1.1 verified. 25 mandatory requirements PASS, 3 recommended requirements PASS.

8 Conclusion

SendCUIEmail v0.17.3 (commit 2048d2e) demonstrates full compliance with REQ-2026-001 v1.1 Cryptographic Compliance Requirements.

Document prepared: January 22, 2026

Verified against commit: 2048d2e4508e83ca77b264949775835a3d3aba96